OPERATING SYSTEMS • CSE • KLH UNIVERSITY

Smart Process Management via Linux System Calls

A comprehensive study engineered through the Linux kernel's native interface — translating OS theory into an interactive demonstration of creation, execution, synchronization, and termination.

IMPLEMENTATION
C & POSIX System Calls

INTERFACE
Interactive CLI Console

PROJECT OVERVIEW

Abstract & Project Scope

The **Smart Process Management System** bridges textbook operating-system concepts with real Linux kernel operations. By wrapping low-level system calls into an interactive console application, users can trigger and observe process lifecycles in real time.

Rather than viewing process management as abstract theory, this application exposes parent-child relationships, PID assignments, signal handling, and clean process reaping directly through native POSIX calls.

PLATFORM
Linux Kernel (POSIX API)

CORE API
fork, exec, wait, kill, getpid

OBJECTIVE
Hands-On OS Education

GOALS & TARGETS

Key Project Objectives

Understand Lifecycle

Observe process state transitions in a live OS environment.

Process Control

Spawn, synchronize, and cleanly terminate processes safely.

Track Hierarchy

Maintain parent-child relationships and monitor process tree.

Interactive CLI

Expose kernel calls via a simple menu-driven interface.

SYSTEM DESIGN

Layered Architecture Model

User Console — Captures user actions and options from the interactive CLI menu.

↓

Process Manager — Translates requests into structured system-call execution chains.

↓

POSIX System Calls — Direct execution of fork(), execvp(), wait(), kill(), and getpid().

↓

Linux Kernel — Kernel schedules processes, manages memory, and controls hardware.

TECHNICAL DEEP DIVE

Core Linux System Calls

`fork()`
Clones calling process. Returns 0 to child, PID to parent.

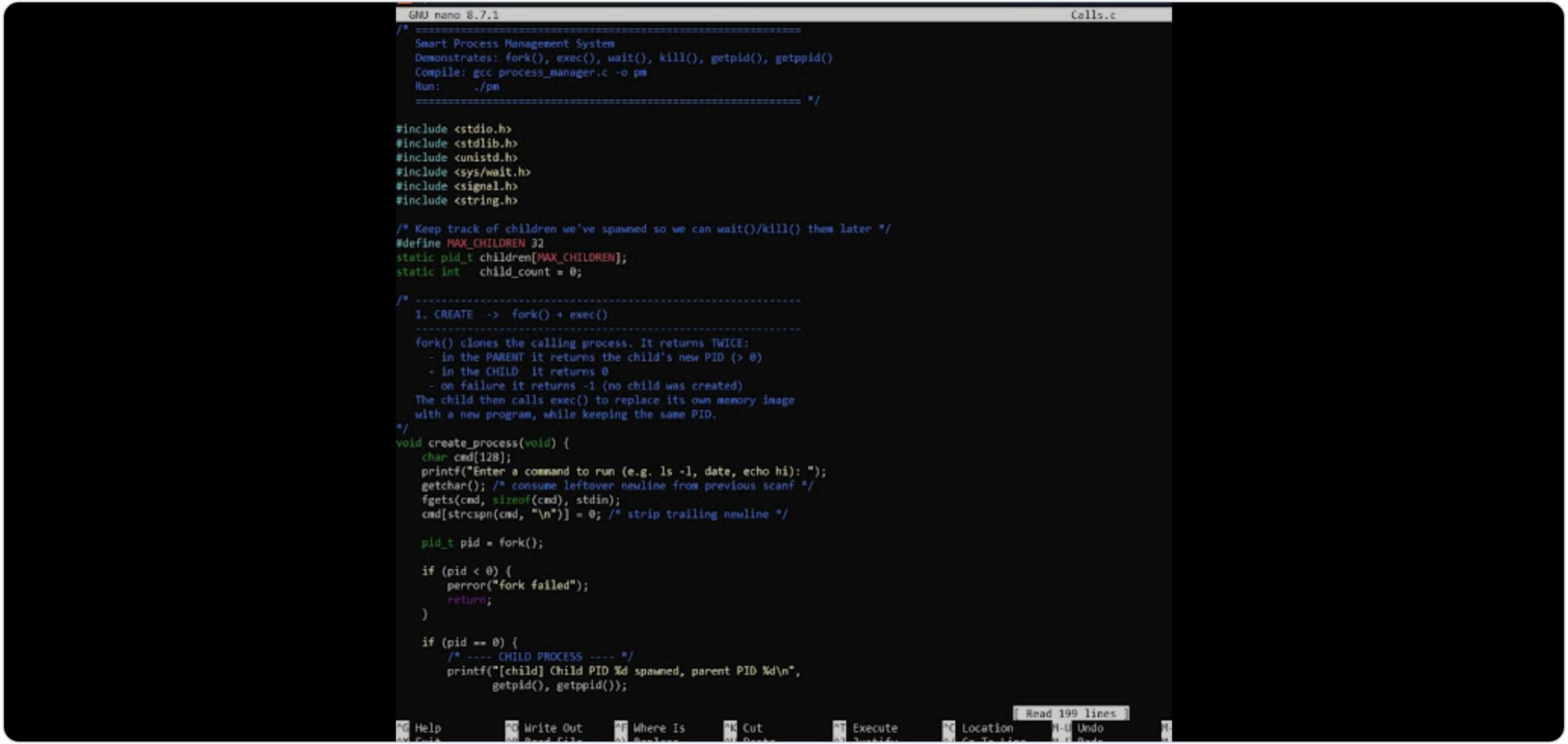
`execvp()`
Replaces process memory image with target executable.

`wait()`
Blocks parent until child exits to prevent zombie state.

`kill()`
Delivers signals to processes for control or termination.

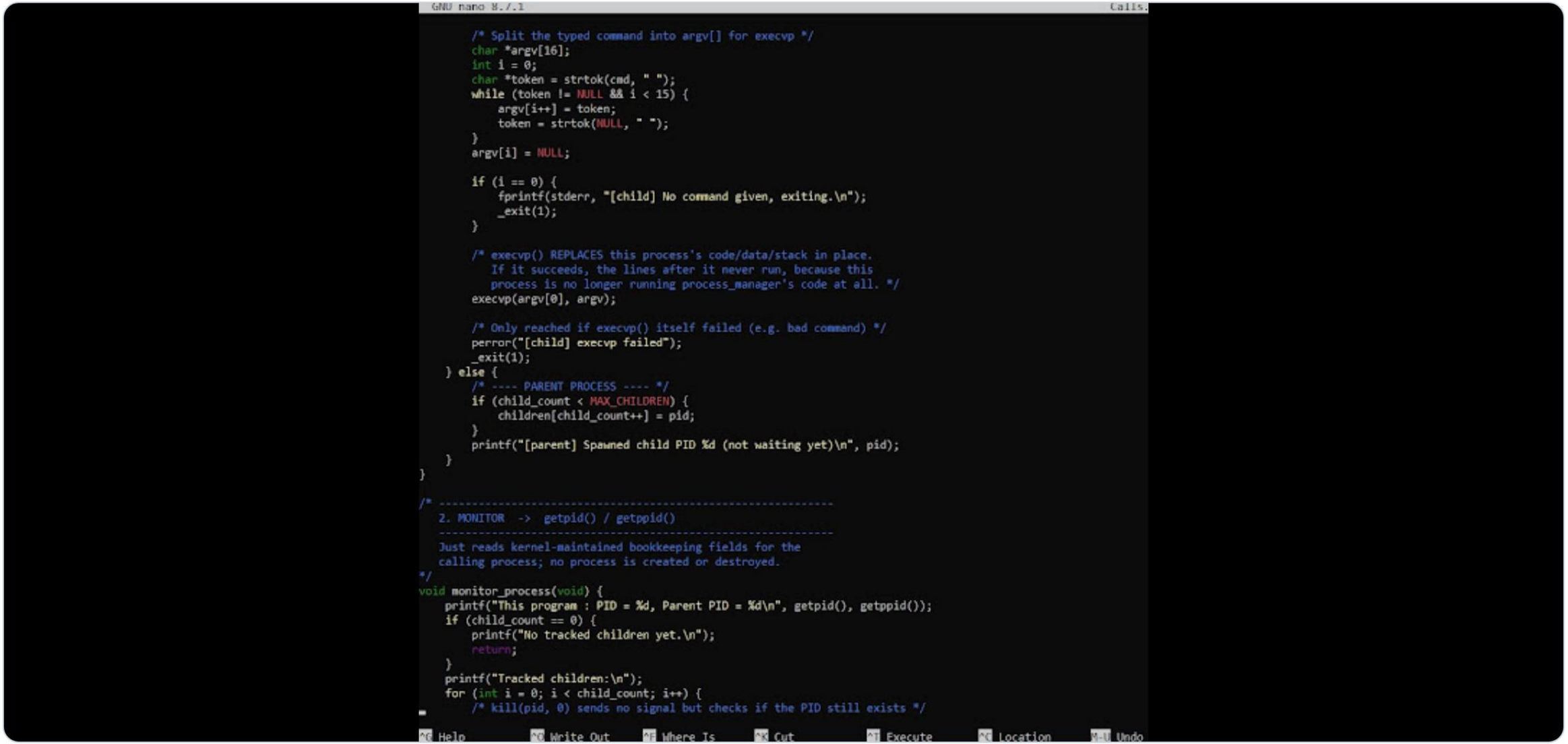
SOURCE CODE SCREENSHOT 1

C Source: Initialization & Forking



SOURCE CODE SCREENSHOT 2

C Source: Command Exec & Monitoring



TERMINAL SCREENSHOT 1

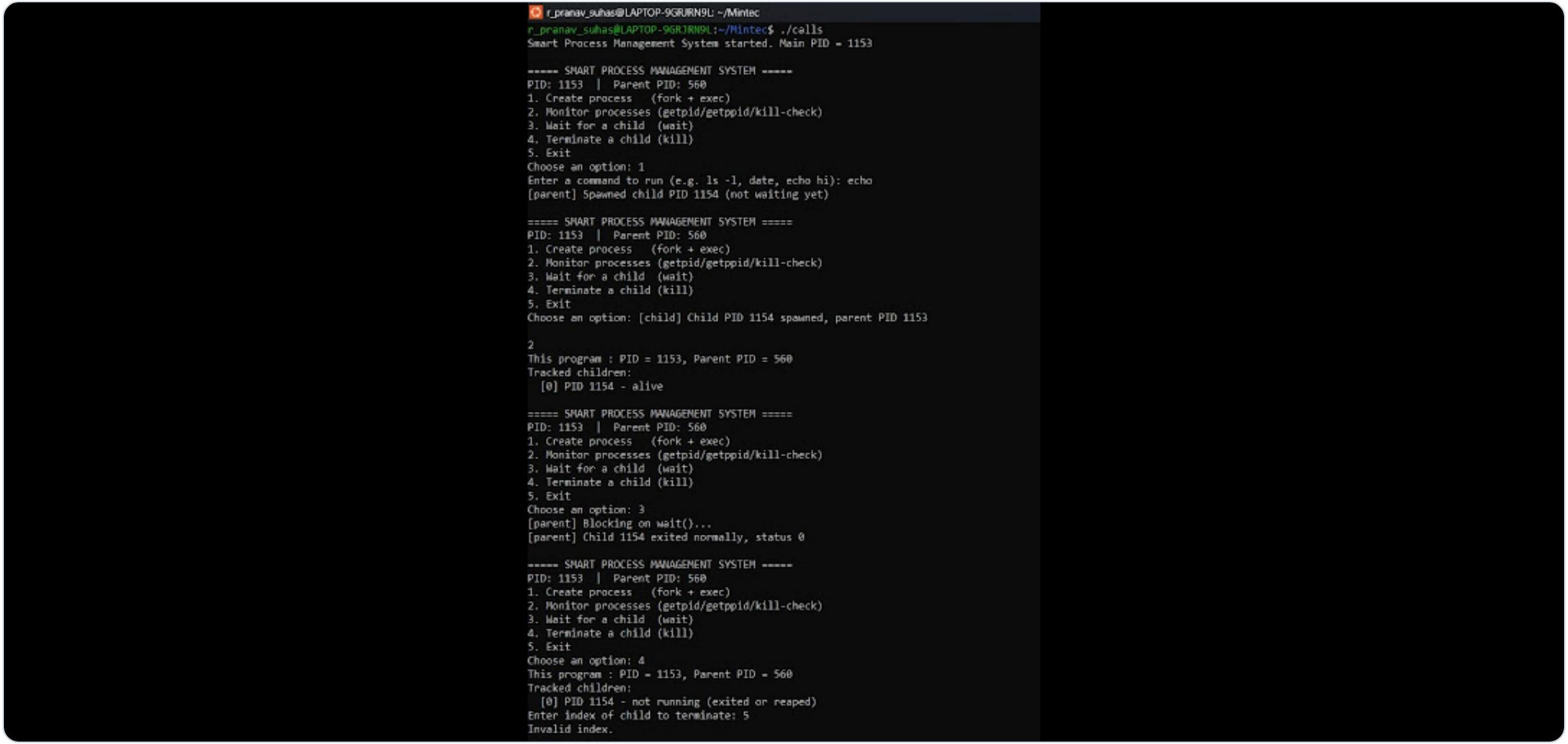
Execution Output: System Startup & Exit

```
r_pranav_suhas@LAPTOP-9GRJRN9L:~/Mintec$ ./calls
Smart Process Management System started. Main PID = 938

===== SMART PROCESS MANAGEMENT SYSTEM =====
PID: 938   |   Parent PID: 560
1. Create process    (fork + exec)
2. Monitor processes (getpid/getppid/kill-check)
3. Wait for a child  (wait)
4. Terminate a child (kill)
5. Exit
Choose an option: 5
Exiting. Reaping any remaining children...
```

TERMINAL SCREENSHOT 2

Execution Output: Interactive Menu Lifecycle



CONCLUSION

Thank You!

Questions & Technical Discussion

KLH University • Department of Computer Science & Engineering